

Processor Design Project (PDP)

CESE4040 (Q4 2025-2026)

Server and Project Manual

Author:

Ignacio Garcia Ezquerro

Table of Contents

1. Server	3
2. RISCY Core	6
3. PDP Project.....	7
3.1 Vivado	7
3.1.1 Project Creation.....	7
3.1.2 Top Level Design	9
3.1.3 Top Level Simulation	10
3.1.4 Synthesis and Implementation	12
3.1.5 Bitstream Generation	14
3.2 Application: AES Code (C).....	15
3.2.1 Compilation and Setup	15
4 FPGA Board.....	16
4.1 Setup	16
4.2 Running on FPGA	17

1. Server

Once the groups are formed an account per group will be provided to work directly in the server. Each project group uses one account on the development server. The server is available at the following address:

ce-procdesign01.ewi.tudelft.nl

Please note that, if you want to connect to the server from outside of the campus network, you have to use [EduVPN](#).

The server provides graphical interface to a Linux based operating system. Therefore, the recommended client software is [X2GO](#). Please download and install the software and make sure that you have carefully read the [QCE guide](#) to setup the connection. See Figures 1, 2 and 3 for more information.

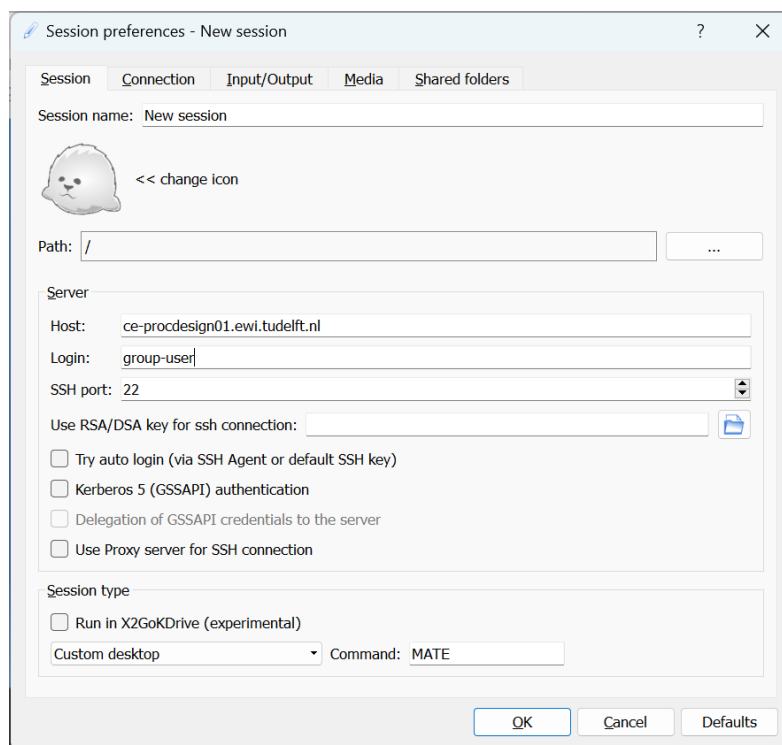


Figure 1. Session window

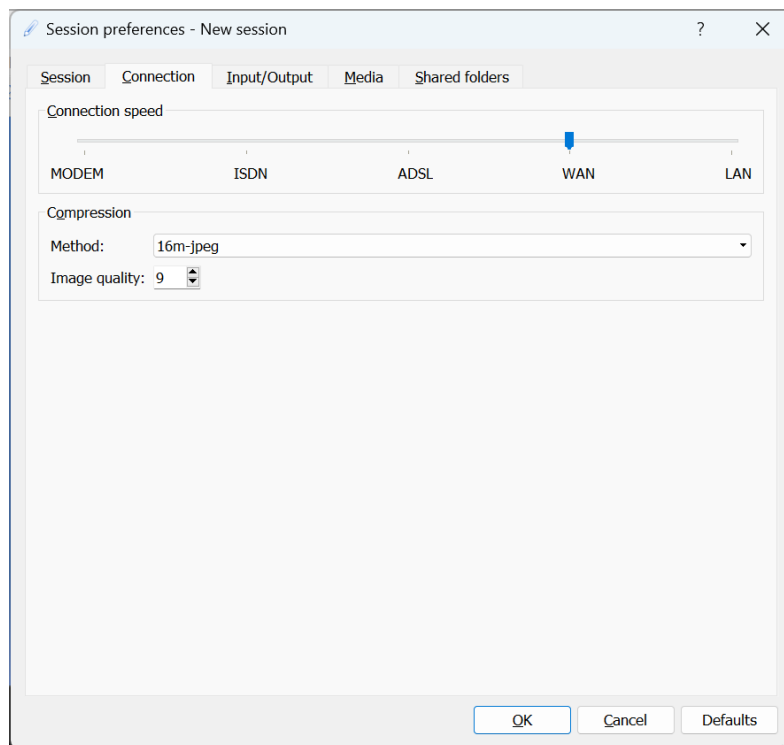


Figure 2.Connection menu

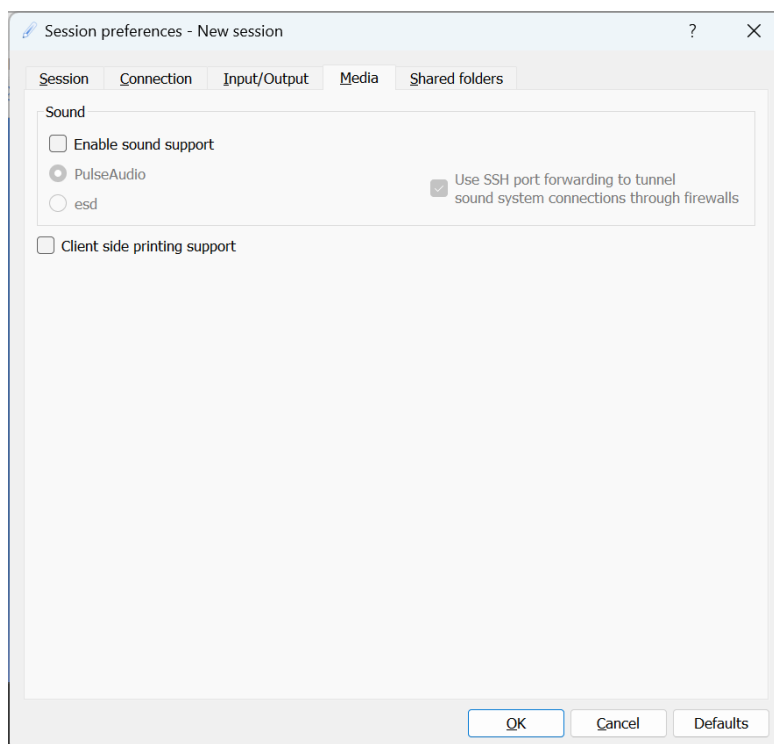


Figure 3. Media menu

Note

It is important to note that when you intend to leave the session, you should always logout from the GUI. Do not close the client window as it will be still running on background and uses unnecessary resources. Administrative settings will terminate any stale session after four hours.

Once you logged in to the session, you can edit sources files via one of the following options:

- Directly from the server using vim or Gedit.
- Remotely from your computer using VSCode and the ssh plugin. See this [guide](#) for more information.

To launch the Vivado Software, open a terminal and type `vivado` as shown in Figure 4.

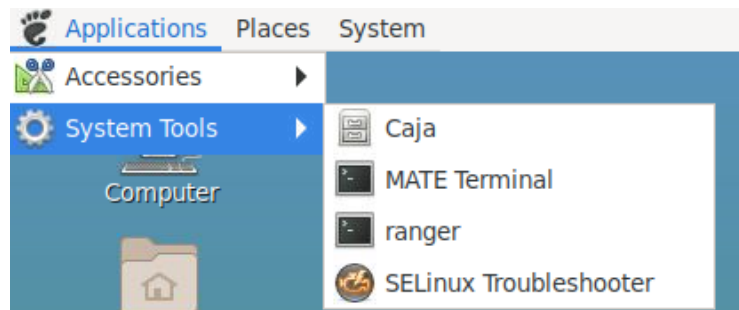


Figure 4. Launching terminal

Note

You should not edit or work in this directory. We recommend you copying both the LLVM and the base project to your home are outside of the course directory. This ensure that you always have the original project files.

2. RISCY Core

[RI5CY](#)¹ is a small four-stage RISC-V core. It implements the RV32IMC base plus extensions (the 32-bit integer ISA, multiply/divide, and compressed instructions), with an optional single-precision FPU (RV32F), and adds the custom Xpulp extensions (hardware loops, post-increment load/store, SIMD, MAC, and others) that improve performance for signal processing workloads. It was developed as part of the [PULP platform](#) for energy-efficient computing. Figure 5 shows the datapath; more detail is in the [manual](#).

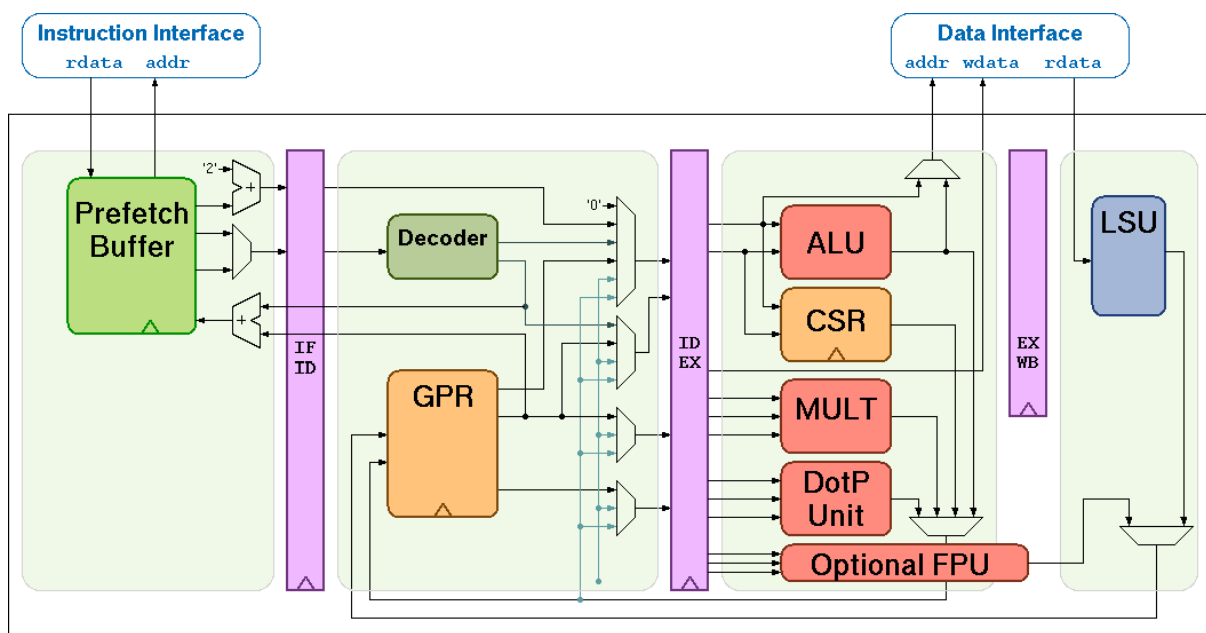


Figure 5. RI5CY datapath

In 2020 the core was contributed to the OpenHW Group and renamed CV32E40P. We still refer to it as RISCY in this course. Several interface and compliance changes were introduced during the handover, so parts of the old PULP documentation are out of date. The current authoritative reference is the OpenHW CV32E40P [user manual](#).

The RTL source files for the RISCY version you will use as the starting point for your project can be found in the course directory in the server, alternatively they can also be found in your group gitlab repository.

¹ In this document, RISCY and RI5CY are used interchangeably.

3. PDP Project

3.1 Vivado

For your convenience, several scripts are provided with the base RISCY repository mentioned earlier. These scripts can be executed directly from the Vivado's tcl console. To do so, first change your directory to `pdp-project/hardware/` then use `source` command as shown Figure 6.

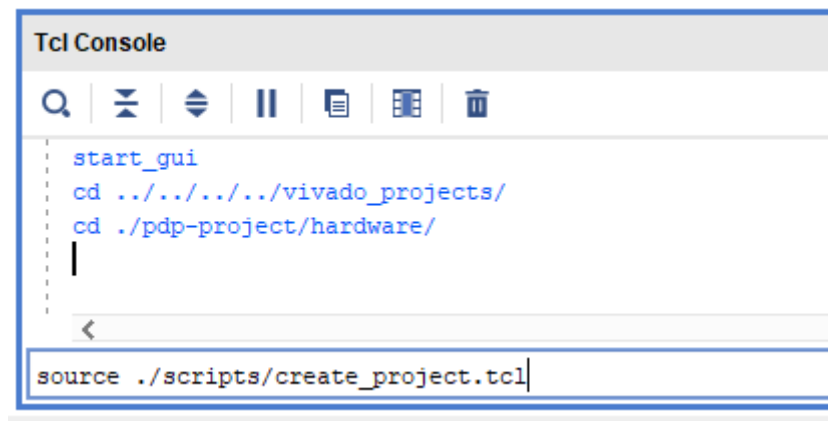


Figure 6. TCL Console

Alternatively, you can execute the script by invoking vivado command with the same effect as Figure 6.

```
vivado -mode batch -source <your_Tcl_script>
```

3.1.1 Project Creation

A dedicated script to generate the base FPGA system that will be used for simulation and implementation. The script is located at `pdp-project/hardware/scripts/create_project.tcl`. After running this script, your project is loaded in Vivado as shown in Figure 7.

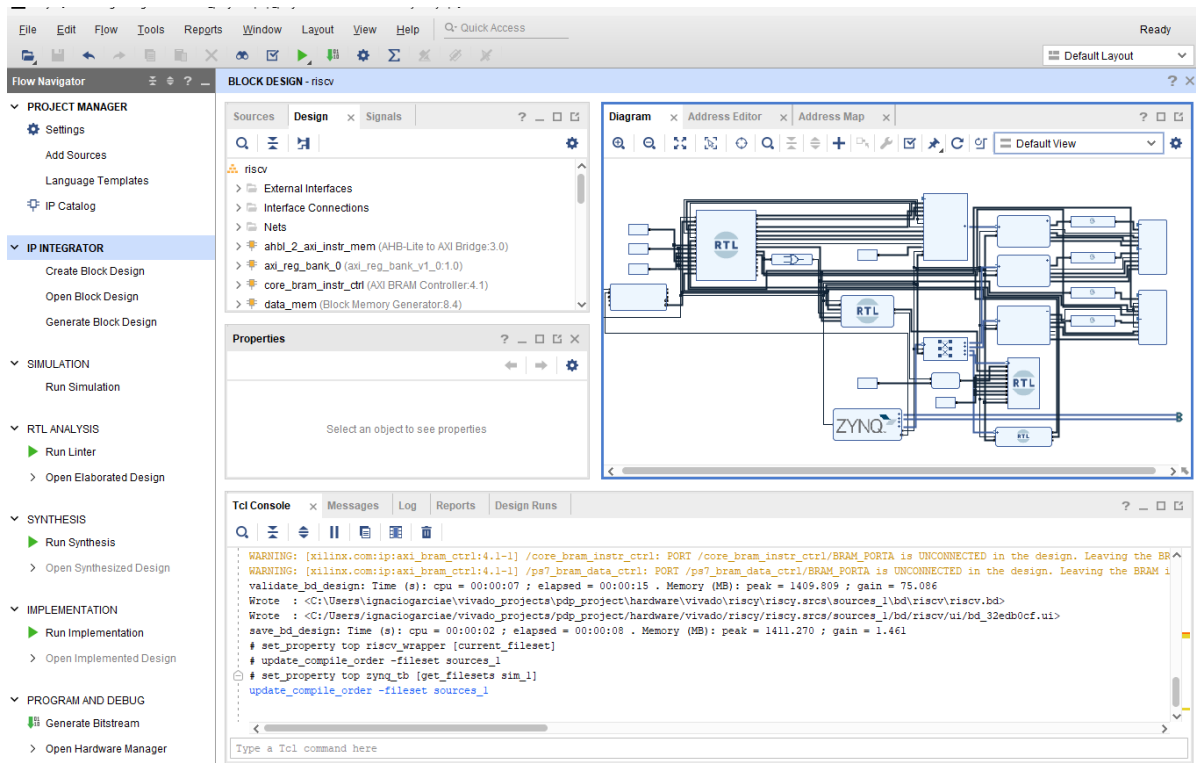


Figure 7. Loaded project in Vivado

Under the *sources* tab, you can find both the RTL source files, constraints and simulation sources, as shown in Figure 8. Selected top levels, often referred to as *tops*, are shown in bold font. There are two *tops* in every design, one for the *Design Sources*, used for synthesis and implementation, and one for the *Simulation Sources*, used for simulation. More information is provided next.

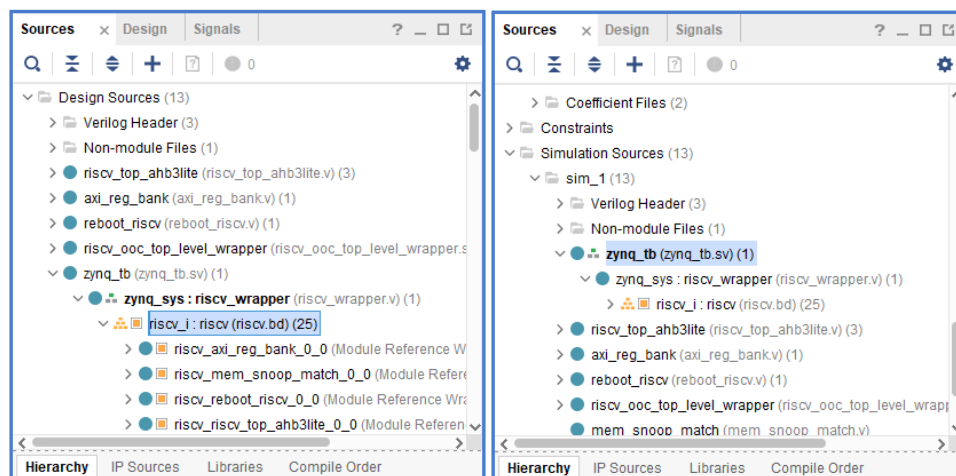


Figure 8. Sources tab

3.1.2 Top Level Design

The design top level source is a block diagram autogenerated using one of the provided TCL scripts in `pdp-project/hardware/scripts/generate_fpga_bd.tcl` and is shown in Figure 9.

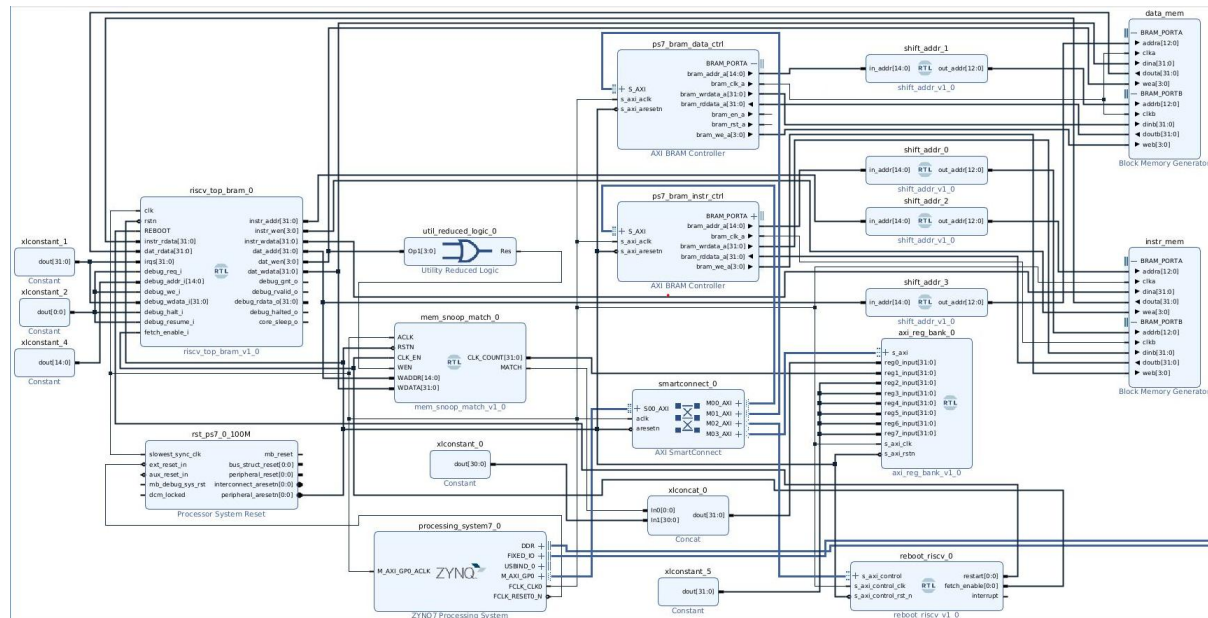


Figure 9. Block diagram of the design

This block diagram shows what will be synthesized and loaded on the FPGA. It consists of the following relevant blocks/modules:

- *riscv_top_bram*: Wrapper around the top level of RISCY, it includes the interfacing IPs used to connect the core to on-chip FPGA SRAM memory blocks, also referred to as BRAMs.
- *instr_mem*: Dual port BRAM containing the instructions executed by your processor.
- *data_mem*: Dual port BRAM containing the data.
- *procesing_system7*: Instance of the hardened ARM core available in the FPGA device, used to generate the clock of your processor design and to write/read data to and from the other IPs, e.g. program and data memories, of the design via an AXI bus.
- *mem_snoop_match*: Small module in charge of snooping the data memory interface, configured to spot the end sequence, 0xDEADBEEF, at address 0x2000. Also counts the clock cycles since its clock enable input is asserted (connected to fetch enable) until the end sequence is found.
- *reboot_riscv*: Control block used in the design to assert/deassert the fetch enable and reboot signals connected to the core.

- *axi_reg_bank*: Register bank used to be able to access PL signals via AXI from the processing system.
- *Interconnection blocks*: Smartconnect (used to access from a single processing system AXI master three AXI slaves), *bram* controllers and so on.
- *shift_addr*: Used given that the generated BRAMs by vivado are addressed per word, instead of per byte as AXI and our core do. Since the word width of the BRAMs is 4-bytes, we just shift the input address by 2 bits.

3.1.3 Top Level Simulation

System simulation:

The testbench source file can be found at `pdp-project/hardware/src/simulation/zynq_tb.sv`.

The testbench just generates the necessary clock and reset signals, connects them to the top level of the design, and performs some reads/writes² via the processing_system AXI.

Both instructions and initial data are preloaded into the BRAMs by the `generate_fpga_bd.tcl` script, where you can check that the memory initialization files being loaded are `code.coe` and `data.coe` and should be placed in `pdp-project/hardware/src/sw/mem_files`.

A script has been provided to run the simulations and preload a selection of waveforms. To use it just run `source ./scripts/run_simulation.tcl`. It will take around a minute to elaborate all the project sources and then it will run the simulation. Depending on the code being executed in the core it will take more or less than four minutes to simulate the baseline software AES encryption. Once the simulation finishes your Vivado window will look like Figure 10.

² Only 13 address bits for the BRAMS are needed, so that module just selects the 14:2 out of the 31:0 generated by the address ports of the core.

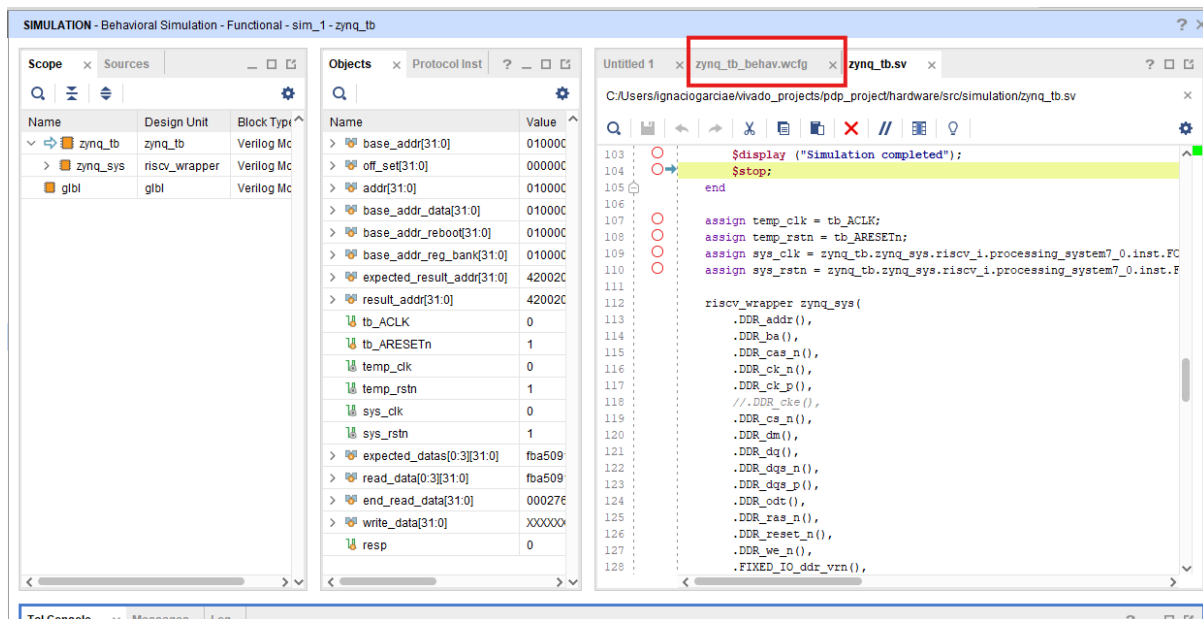


Figure 10. Simulation output

The waveform window with the preselected signals is highlighted in red. You can always add or modify the selected signals and save your view. This way the next time you run a simulation with the script it will show your own version.

If you add signals that were not selected before running the simulation, you will only be able to see their final value. Click on *restart*, and then on *run* as shown in Figure 11.

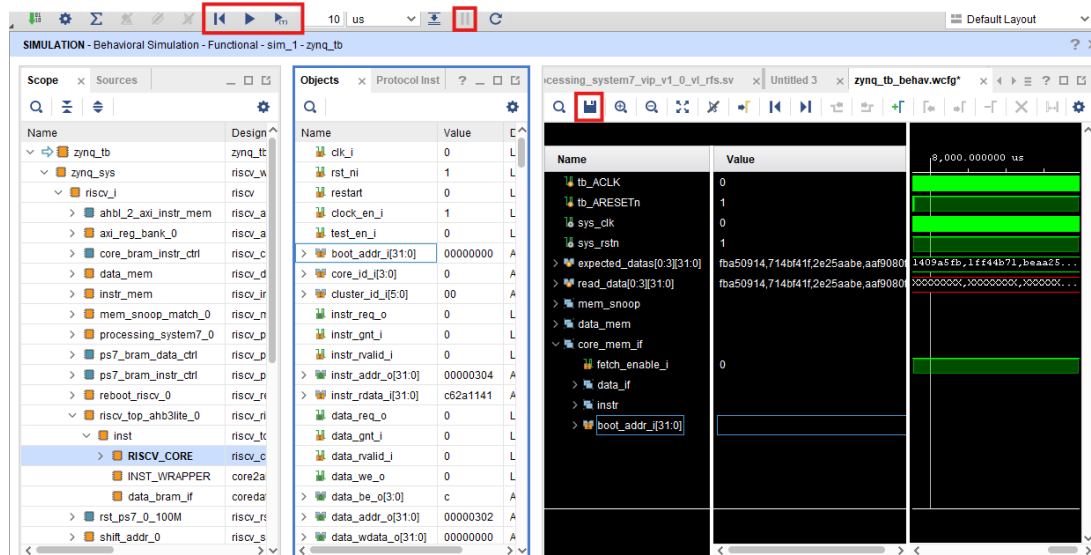


Figure 11. Waveform output

You can always stop the simulation at whatever point you want by using the *pause* button. The console output of the base project by the end of the simulation is shown in Figure 12.

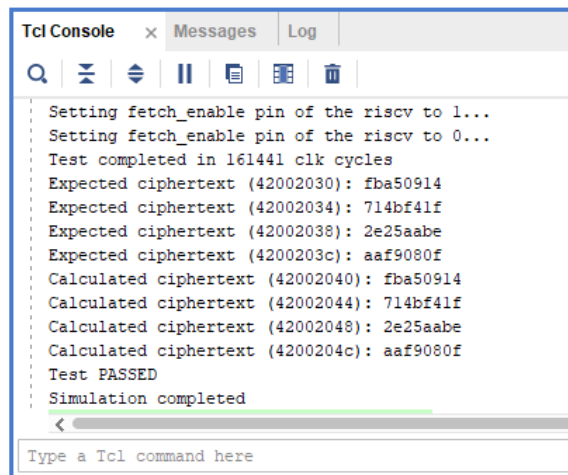
The image shows a screenshot of a software interface with a 'Tcl Console' window. The window has a title bar with 'Tcl Console', 'Messages', and 'Log' tabs. Below the title bar is a toolbar with icons for search, zoom, run, pause, save, print, and delete. The main area of the console displays the following text: 'Setting fetch_enable pin of the riscv to 1...', 'Setting fetch_enable pin of the riscv to 0...', 'Test completed in 161441 clk cycles', 'Expected ciphertext (42002030): fba50914', 'Expected ciphertext (42002034): 714bf41f', 'Expected ciphertext (42002038): 2e25aabe', 'Expected ciphertext (4200203c): aaf9080f', 'Calculated ciphertext (42002040): fba50914', 'Calculated ciphertext (42002044): 714bf41f', 'Calculated ciphertext (42002048): 2e25aabe', 'Calculated ciphertext (4200204c): aaf9080f', 'Test PASSED', and 'Simulation completed'. At the bottom of the console is a text input field with the placeholder text 'Type a Tcl command here'.

Figure 12. Console output after simulation finishes

Module simulation:

Only testbench files and scripts for system level simulation are provided (core + memories), if lower-level testing is required (i.e: modules that are part of the core) you are expected to implement all of the required files (testbench, testcases, scripts) needed to verify and debug your project yourself.

3.1.4 Synthesis and Implementation

If you just want to verify your changes to the RISCY core or how does the resource utilization changed, it is recommended to just run an OOC synthesis³ as it will be faster than running synthesis on the entire project.

To run the OOC synthesis just open a new vivado window, change directory to the hardware directory and then source this script `pdp-project/hardware/scripts/create_project_ooc_synth.tcl`. This script will also save the results in `pdp-project/hardware/vivado/ooc_riscy/ooc_riscy.runs/ooc_synth/`. In Vivado the reports look like Figure 13.

³ Out Of Context (OOC) Synthesis is a technique used in FPGA design flows, where individual modules or IP blocks are synthesized separately from the top-level design. This is helpful when performing sanity checks to the RISCY core is required without synthesizing the whole FPGA design.

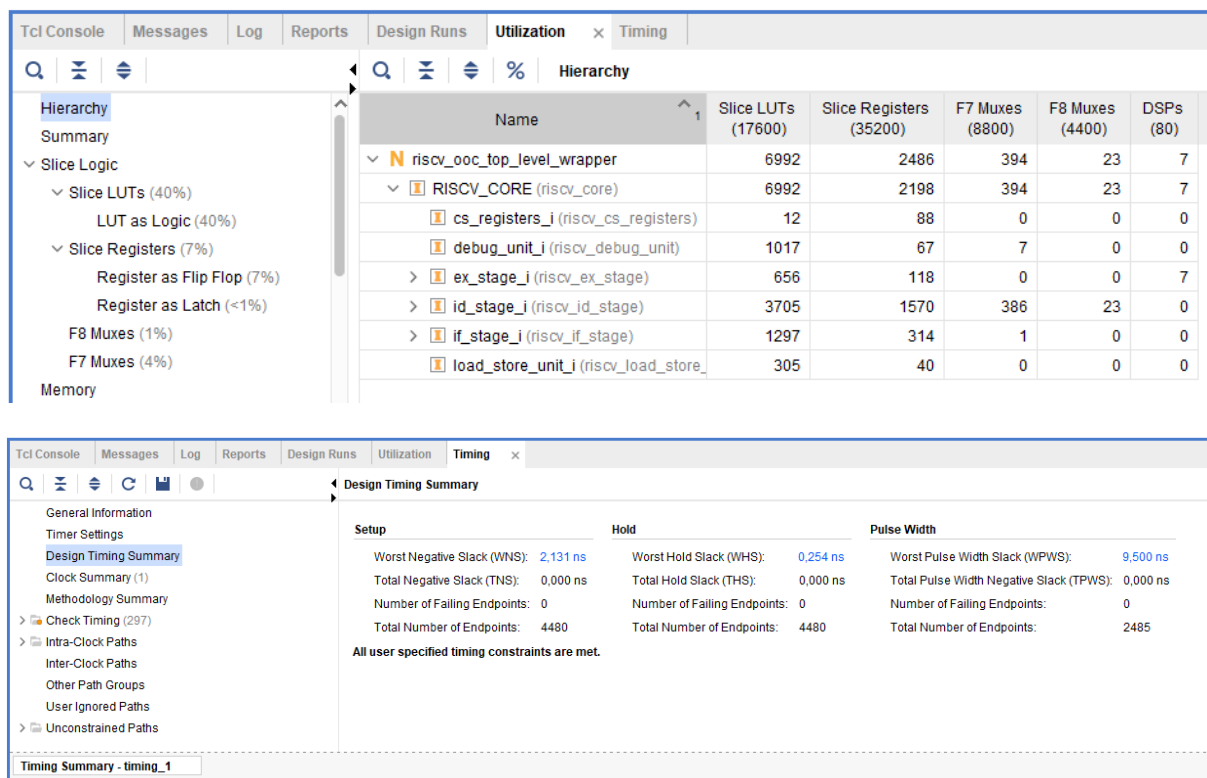


Figure 13. Timing and utilization reports

To generate these reports manually for any design, just click on *Open Synthesized* or *Open Implemented Design* depending on the step you got and then on *Reports->Timing* or *Reports->Report Utilization* Utilization/Timing as shown in Figure 14.

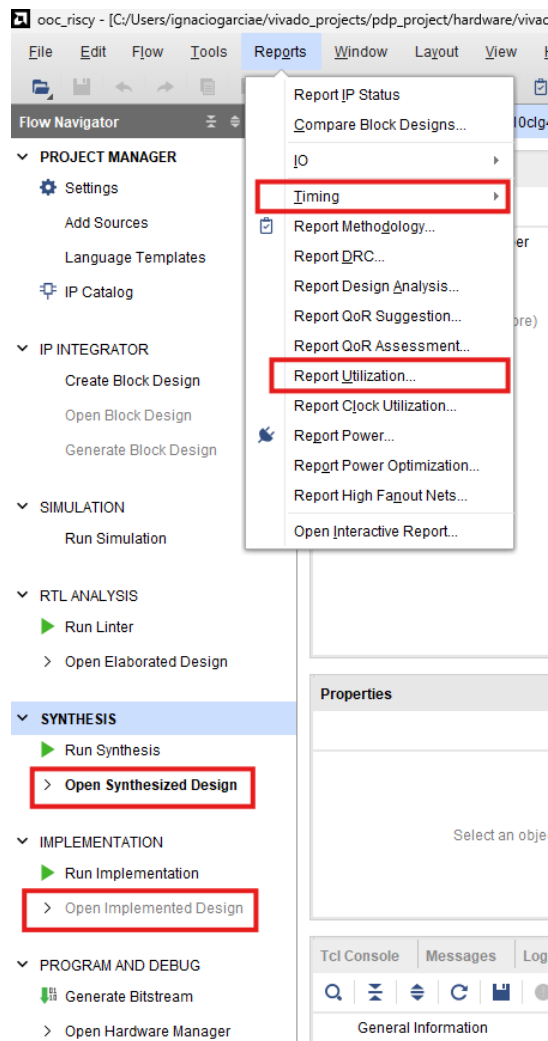


Figure 14. Reports menu

If you want to run synthesis or implementation for the full project manually, you can always use the *Run Synthesis* or *Run Implementation* options.

3.1.5 Bitstream Generation

You can generate the FPGA bitstream from your RISCY project by just sourcing `pdp-project/hardware/scripts/run_synth_impl.tcl`. You can also create the project and generate the bitstream with a single script by sourcing `pdp-project/hardware/scripts/gen_bitstream.tcl` from a new vivado window. You can find the generated bitstream at `pdp-project/hardware/vivado/riscy/riscy.runs/impl_1/riscv_wrapper.bit`.

Note

It is always advised to generate timing and utilization resources of the implemented design before trying to use the bitstream on the board, this way you can spot any possible timing violations or RTL getting optimized away.

3.2 Application: AES Code (C)

The baseline software implementation of the AES encryption is provided in C and can be found in `pdp-project/software/main.c`.

This source file contains all the functions used to perform the encryption, the initial text, the key to be used and the expected ciphertext. For example, the input text “Hello, World!000” and the key “cese4040password” are hardcoded in the program.

The main function of this C file just performs the encryption of the input text, writes the ciphertext and the expected ciphertext to a given address, checks the output and writes either 0xBAAAAAAD or 0xCADEBABA depending on if the ciphertext matches expected, and writes the end sequence 0xDEADBEEF detected by the *mem_snoop* module on the FPGA to a specific address 0x0102000.

Note

The ending sequence should not be removed, as it is used by the testbench to know when the core finishes with the execution of the main and stop the simulation.

3.2.1 Compilation and Setup

It is first necessary to update the paths within the `pdp-project/software/config/rv32-standard.conf` to point to the specific directories where the RISC-V GCC toolchain and LLVM are located.

To do this modify, if needed, the paths assigned to the variables `RISCV_GCC` and `LLVM`.

To compile the C code and generate the executable binary, and also the memory initialization files used by the FPGA project, it is only needed to run `make soft` within the `pdp-project/software` directory. This will create the binary in `pdp-project/software/output/soft.elf` and the memory initialization files (`.coe`) files in `pdp-project/software/bin_files`. These generated `.coe` files need to be copied if the path is changed to the FPGA project, `pdp-project/hardware/src/sw/mem_files`, so they can be picked up by the simulation flow.

4 FPGA Board

4.1 Setup

This project uses the PYNQ-Z1 Board from Digilent. It has the ZYNQ-7000 SoC device from AMD (formerly Xilinx) with a hardened ARM Cortex-A9 core directly connected to the Programmable Logic (PL). The communication with the board will be done via ethernet cable, which will be used to run Jupyter Notebooks in the processing system (ARM core). To be able to do this, the board needs to be set up as shown in Figure 15. For more information, see the board [manual](#).

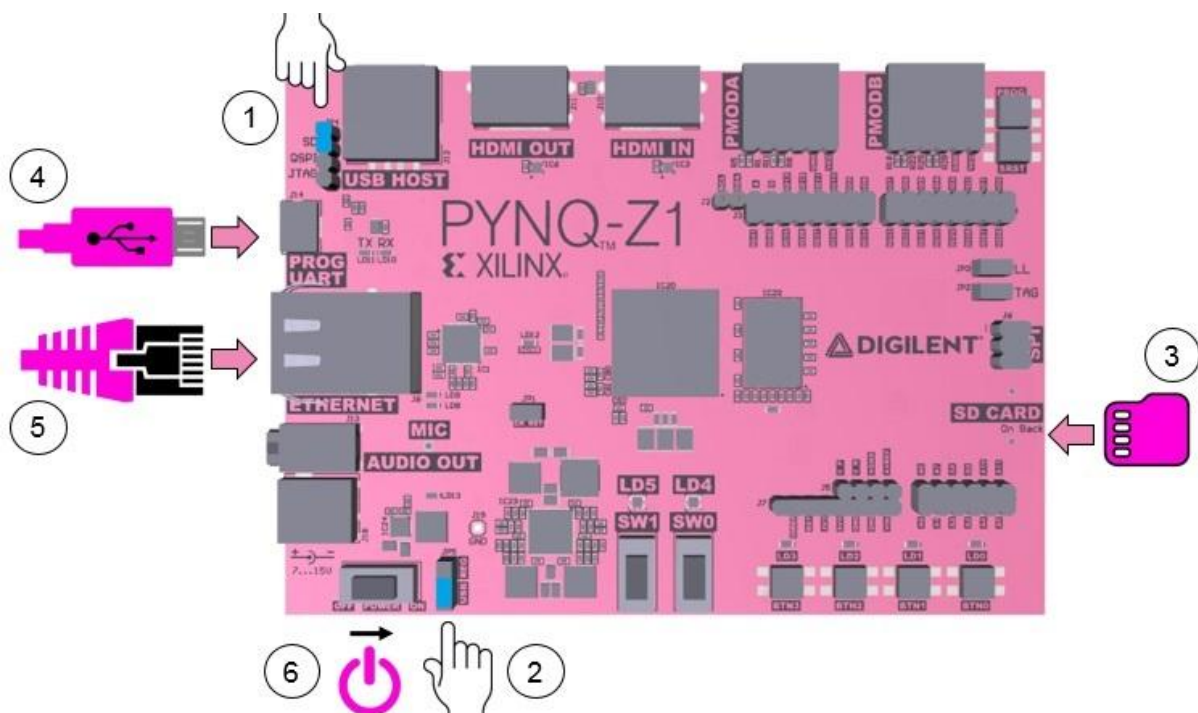


Figure 15. PYNZ-Z1 board

Follow the steps below:

1. Set the JP4 / Boot jumper to the SD position by placing the jumper over the top two pins of JP4. This sets the board to boot from the Micro-SD card.
2. To power the PYNQ-Z1 from the micro USB cable, set the JP5 / Power jumper to the USB position. You can also power the board from an external 12V power regulator by setting the jumper to REG.
3. Insert the Micro SD card loaded with the PYNQ-Z1 image into the Micro SD card slot underneath the board.
4. Connect the USB cable to your PC/Laptop, and to the PROG - UART / J14 MicroUSB port on the board.
5. Connect the board to Ethernet by following the instructions below.
6. Turn on the PYNQ-Z1 and check the boot sequence (explained below).

Boot sequence:

Slide the power switch to the ON position to turn on the board. The Red LD13 LED will come on immediately to confirm that the board has power. After a few seconds, the Yellow/Green LD12 / Done LED will light up (green) to show that the ZYNQ device is operational.

After a minute you should see two Blue LD4 & LD5 LEDs and four Yellow/Green LD0-LD3 LEDs flash simultaneously. The Blue LD4-LD5 LEDs will then turn on and off while the Yellow/Green LD0-LD3 LEDs remain on. The system is now booted and ready for use.

Connect the FPGA board to a computer:

You need a computer with a free Ethernet port⁴ ⁵. You also need to have the permissions to configure your network interface. With a direct connection, you are able to use PYNQ.

Follow the steps below to connect directly to a computer using static IP⁶ address:

1. Assign a static IP address ([example](#)).
2. Connect the board to your computer's Ethernet port.
3. Open a web browser and browse to <http://192.168.2.99>.
4. The credentials to login on the web page are xilinx/xilinx.

4.2 Running on FPGA

Once logged in, the main screen of the Jupyter notebook is similar to Figure 16. If you are not familiar with Jupyter notebook, you may want to check [this](#) tutorial.

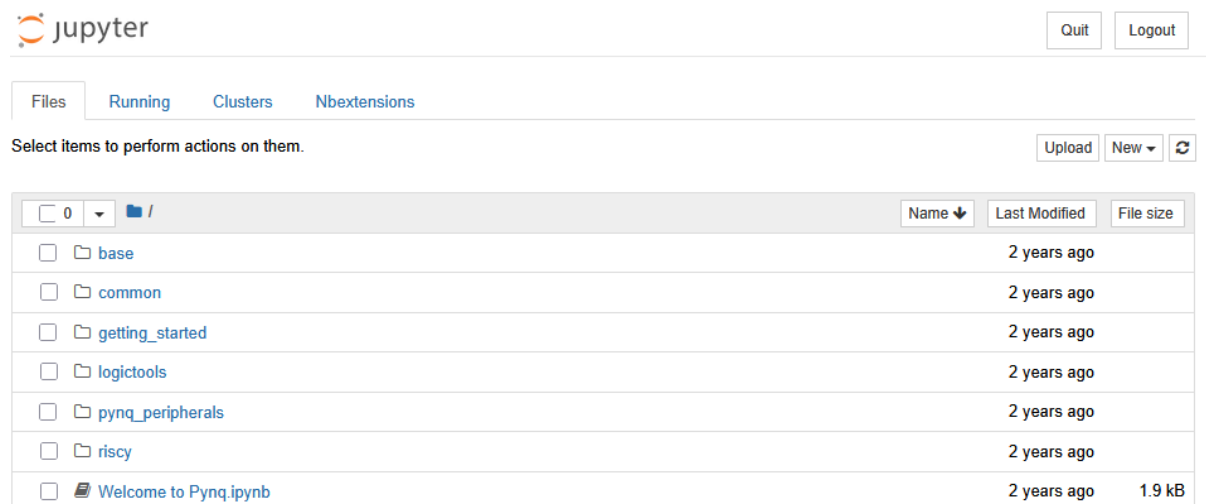


Figure 16. Jupyter notebook web page

⁴ If you don't have the cable, you may ask for it in the weekly meetings.

⁵ If your laptop has no Ethernet port, you need an Ethernet-to-USB converter. You may ask for it in the weekly meetings.

⁶ Here, IP refers to Internet Protocol and not Intellectual Property.

The content of the folder `riscy` is shown in Figure 17.



	Name	Last Modified	File size
..	..	seconds ago	
mem_files	mem_files	2 years ago	
overlays	overlays	2 years ago	
base_riscy.ipynb	base_riscy.ipynb	Running 2 years ago	15 kB

Figure 17. Folder content

Inside `mem_files` you see `data.coe` and `code.coe`, which are the memory initialization files for the BRAMs connected to the RISCY core.

The `overlays` directory contains the following:

- `base_riscy.bit`: Generated bitstream from the main `riscy` project, can be found in `pdp-project/hardware/vivado/riscy/riscy.runs/impl_1/riscv_wrapper.bit`.
- `base_riscy.tcl`: Generated at the same time as the bitstream, you can find it in `pdp-project/hardware/vivado/riscy/riscy.runs/impl_1/riscv_wrapper.tcl`.
- `base_riscy.hwh`: Hardware handoff file, autogenerated from the `riscy` project in `pdp-project/hardware/vivado/riscy/riscy.gen/sources_1/bd/riscv/hw_handoff/riscv.hwh`.

These 3 files will have different names when you generate them using Vivado. Then you should rename them to the same to be able to use them in the Jupyter Notebook and the name should match that of the bitstream when importing it with the *Overlay* command inside the Jupyter notebook.

You can find a copy of this base directory in `pdp-project/hardware/sw/fpga/riscy`. With all the files available and accessible via the browser in the FPGA, you can now proceed to run the `base_riscy.ipynb` notebook.

This notebook contains the code and explanations to perform the following actions:

1. Initialize the notebook, load the overlay (PL image/bitstream) and some helper functions to initialize the BRAMs.
2. Program the PL with the bitstream.
3. Control the RISCY core by performing AXI writes/reads to the *riscv_control* module.
4. Programming the instruction and data memories.
5. Read the instruction and data memories.
6. Start/stop the fetching from the core.
7. Check the results written by the core to memories or the duration of the test registered by the *mem_snoop* module.